

# Lec10 Container-stack-queue

- Overview
  1. Sequence containers  
vector / deque / list / forward\_list / array / string
  2. Iterator and iterator range  
begin(), end(), [begin, end)
  3. Add / remove elements  
push\_back, push\_front, insert, erase, resize
  4. Container adapters  
stack / queue / priority\_queue

## 选择container的规则

1. vector 默认使用
  - contiguous memory locations
    - 支持随机访问，容易push\_back
    - 中部insert难
2. list & forward\_list 双向/单向链表
  - 不支持random access `list[3]` 是错误的
  - 适合经常在中间插入删除
3. deque (double-ended queue)
  - 在两端可以高效地插入删除 `pop_front, push_front`
4. array & string

## list&forward\_list

1. 实现singly\_linked list

```
1.  template <typename T>
    struct Node{
        T data;
        Node<T>* next;  ← Node<T> 类型的ptr, 尽管这个struct没有完整命名完, 允许声明这个指针
        Node (const T& value) :
            data(value), next(nullptr) {}
    };

```

C++

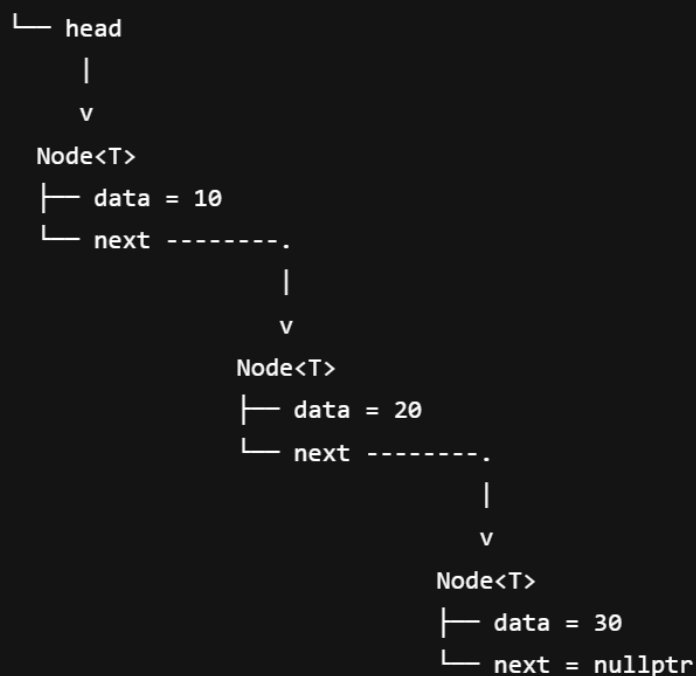
```
template <typename T>
class SinglyLinkedList{
public:
    SinglyLinkedList() : head (nullptr) {}
    void inset(T data);
    void print() const;
private:
    Node<T>* head;
}
```

- list 本体不存数据，data在node里面

C++

```
template <typename T>
void SinglyLinkedList<T>::insert(T data){
    Node<T>* newNode = new Node<T>(data);
    if (head == nullptr){
        head = newNode;
        return;
    }
    Node<T>* current = head;
    while (current->next != nullptr){
        current = current->next; ←一直把current移动到最后一个
    }
    current->next = newNode; ←当current->next = nullptr,赋值
}
```

SinglyLinkedList



- 注意head 和 next的区别和关系

## iterator ranges

- `begin == end`: empty
- `begin != end`: at least 1 element(`*begin`)
  - 所以会用到这个判断: `if (it != numbers.end())`

## assignment and swap

```
vector<int> vec {2,3,5};  
vector<int> copy(vec);
```

C++

- 只有container的类型和变量类型都一样的时候才能这样

```
vector<int> vec {2,3,5};  
deque<int> copy2(vec.begin(),vec.end()); 2,3,5  
vector<double> copy3(vec.rbegin(), -- vec.rend()); 5,3 ← 没有2
```

C++

- container类型或者元素类型不同

```
c3.assign(vec.begin(),ve.())
```

- 完全相同可以swap

```
c1.swap(vec)
```

## 关系运算符relational operators

- 所有container都支持 `== !=`
- 大小比较
  - 从头开始比大小, 完全一样, 短的小

## access elements

- `forward_list`
  - 不方便进行 back 相关的操作 (要从head一直到back)

### 1. remove

1. `push_back()` / `emplace_back()`

1. 除了array和forward\_list

2. `push_front()` / `empalce_front()`

1. 除了vector, array (deque, list, forward\_list可以)

### 3. erase(iter)

1. 会返回下一个iter

### 4. erase(b,e)

1. 删除[, )内的元素

## 2. insert

## Iterator Invalid

- 如果要erase/insert 注意iter的失效  
eg.正确的写法

```
for (auto it = container.begin() ; it != container.end();){  
    if (need_remove){  
        it = container.erase(it);  
    }  
    else{  
        ++it;  
    }  
}
```

C++

## stack

## queue

## priority\_queue

- priority + less : 最大的在top（默认情况）（top先出，也许可以理解为queue的front）
- priority + greater : 最小的在top
- std::pair<1,2>: 1, 2是两种数据类型。先比较1再比较2